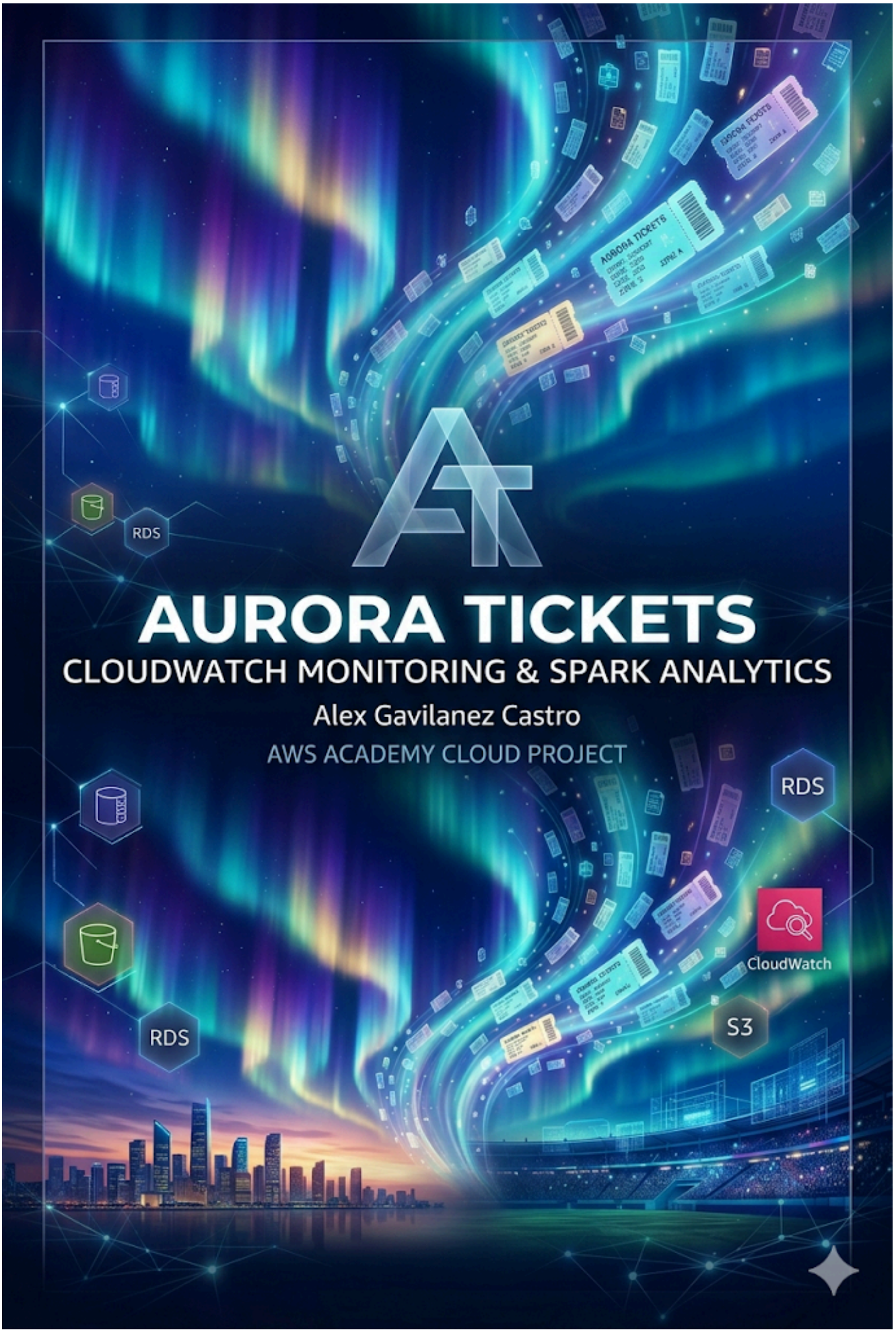

RDS
AURORA TICKETS
CLOUDWATCH MONITORING & SPARK ANALYTICS
Alex Gavilanez Castro
AWS ACADEMY CLOUD PROJECT
RDS
CloudWatch
RDS
S3

Índice

Índice 2
1. Business Understanding 4
1.1 Contexto del Proyecto 5
1.2 Objetivos de Negocio 5
1.3 KPIs 5
1.4 Planificación 5
2. Data Understanding 6
2.1 Origen de los Datos 6
2.2 Estructura del Log (Clickstream) 6
2.3 Volumen y Calidad 6
3. Data Preparation 6
3.1 Crear el Bucket de S3 (El "Data Lake") 6
3.2 Creación de la estructura de capas 6
3.3 Configurar Credenciales en el nodo Web 7
3.4 Generación de los 200,000 registros 8
3.5 Creación de datos extra 10
4. Modeling 12
4.1 Crear un Security Group: 12
4.2 Lanzar las 6 instancias EC2: 13
4.2.1 Creación de script de instalación 13
4.2.2 Crea la AMI: 14
4.2.3 Lanza el resto: 14
4.2.4 Etiquetar las instancias: 15
4.3 Iniciación de los clusters Master y Workers: 15
5. Evaluation and storage 17
5.1 Preparar el Spark-Master 17
5.2 Configurar las credenciales en el Master 17
5.3 Asignación de IAM Roles 17
5.4 Creación del Script de procesamiento 18
5.5 Crear la Base de Datos RDS en AWS 20
5.6 Creación de la base de datos aurora_db 20
5.7 Preparar el "Job de Analítica " de Spark 21
6. Deployment 23
6.1. Preparación del Entorno 23
6.2. Creación de la API de Consulta (main.py) 24
6.3. Ejecución del Servidor Web 25
6.4. Exposición de KPIs mediante API REST distribuida 25
6.5 Observabilidad con CloudWatch 26
6.5.1 Instalar el Agente de CloudWatch (En el Master y Web Server) 26
6.5.2 Configurar el envío de logs 26
6.5.3 Crear los archivos de Log 28
6.5.4 Arrancar el Agente (En ambos) 28
6.5.5 Generar "Ruido" (Datos para ver en CloudWatch) 28
6.5.6 Verificar en la Consola de AWS 29
6.5.7 Las 4 Consultas de Insights 29
6.6 Creación del Dashboard 31

1. Business Understanding

1.1 Contexto del Proyecto

Aurora Tickets es una plataforma de venta de entradas para eventos que ha experimentado un crecimiento descontrolado. Actualmente, la empresa sufre de falta de visibilidad sobre el comportamiento de sus usuarios, posibles ataques de bots que agotan el inventario y errores técnicos en el proceso de pago que resultan en pérdidas económicas directas.

1.2 Objetivos de Negocio

El objetivo principal es transformar los datos brutos de navegación (*clickstream*) en información accionable para:

- **Optimizar el Embudo de Ventas:** Identificar en qué paso (selección, datos, pago) abandonan más usuarios.
- **Detección de anomalías:** Identificar patrones de tráfico no humano (bots) que saturan el sistema.
- **Estabilidad Técnica:** Monitorizar en tiempo real los errores del servidor para reducir el tiempo de respuesta ante caídas.

1.3 KPIs

Para medir el éxito del proyecto, se han definido los siguientes indicadores que se visualizarán en el Dashboard de CloudWatch y en el análisis final:

1. **Conversion Rate:** Porcentaje de sesiones que terminan en una compra confirmada.
2. **Revenue per Ticket Type:** Ingresos totales desglosados por categorías de entrada (A, B y C).
3. **Bot Traffic Ratio:** Porcentaje de peticiones sospechosas identificadas mediante el análisis de logs.
4. **Error Rate por Endpoint:** Frecuencia de errores en la API de pagos y el servidor web para asegurar un SLA alto.

1.4 Planificación

Para lograr estos objetivos, se desplegará una arquitectura de Big Data en AWS compuesta por:

- Un clúster de **Apache Spark 3.5.x** (1 Master, 3 Workers) para el procesamiento distribuido.
- Un **Data Lake en S3** organizado en capas (Raw, Curated, Analytics).
- **CloudWatch Logs** para la ingesta de eventos en tiempo real.
- **RDS (MySQL)** para el almacenamiento de los resultados agregados finales.

2. Data Understanding

2.1 Origen de los Datos

Los datos son generados por un simulador de tráfico que emula el comportamiento de usuarios reales y bots en la web de "Aurora Tickets". Estos eventos se capturan inicialmente en los logs del servidor web (EC2-6) y se envían a **CloudWatch Logs**.

2.2 Estructura del Log (Clickstream)

Cada evento de navegación sigue un formato JSON (o semi-estructurado en log) con los siguientes campos clave que debemos documentar:

- **timestamp**: Fecha y hora del evento (formato ISO 8601).
- **user_id / session_id**: Identificadores únicos para rastrear el embudo de ventas.
- **method**: Acción realizada (GET/POST).
- **endpoint**: La página visitada (`/home`, `/tickets`, `/checkout`, `/pago_confirmado`).
- **status_code**: Respuesta del servidor (200 OK, 404 Not Found, 500 Error).
- **ticket_type**: Categoría de la entrada si el evento es una selección (A, B o C).
- **ip_address**: Vital para identificar patrones de bots (muchas peticiones desde una sola IP).
- **user_agent**: Navegador/dispositivo (ayuda a distinguir humanos de scripts).

2.3 Volumen y Calidad

- **Volumen:** ~210,000 registros (simulando una semana de alta demanda).
- **Calidad:** Los datos pueden contener nulos en `ticket_type` (cuando el usuario solo mira la home) y errores 500 que interrumpen el flujo. Identificar estos "huecos" es parte del análisis.

3. Data Preparation

Creación del almacén de datos (S3) y llenarlo con la información de "Aurora Tickets".

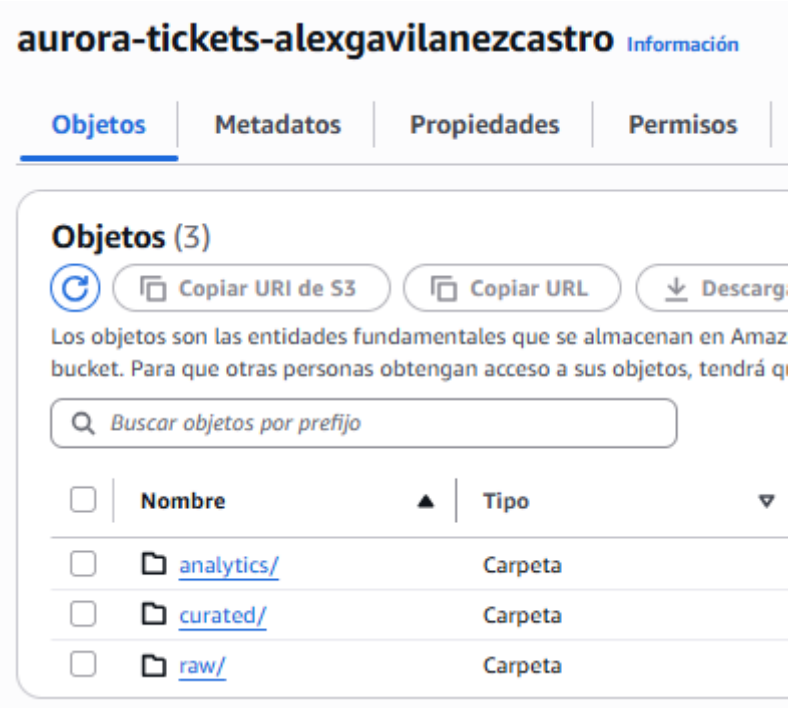
3.1 Crear el Bucket de S3 (El "Data Lake")

- Entró en la consola de AWS y busco S3.
- Hacemos click en Create bucket.
- Bucket name: aurora-tickets-alexgavilanezcastro
- Dejo todo lo demás por defecto y Create bucket.

3.2 Creación de la estructura de capas

Una vez creado el bucket. Vamos a crear las 3 carpetas que definen el ciclo de vida del dato:

1. En el S3 que acabamos de crear ,hacemos click en **Create folder**.
2. Folder name: raw (aquí irán los CSV tal cual se generan). Pulso **Create folder**.
3. Repito los mismo para crear una carpeta llamada curated (aquí Spark guardará los datos limpios) y una carpeta llamada analytics (aquí irán los resultados finales).



3.3 Configurar Credenciales en el nodo Web

Para que nuestra instancia Aurora-Web-Server pueda subir los 200,000 registros al S3, necesita "permiso".

1. En la consola de AWS Academy, busco el botón **AWS Details**.
2. Y donde dice **AWS CLI** copio el bloque de texto que contiene:
   - aws_access_key_id
   - aws_secret_access_key
   - aws_session_token
3. Ahora, entro por SSH a mi instancia **Aurora-Web-Server** y ejecuto:

Para instalar awscli :

```
# 1. Descargar el zip oficial de AWS
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
```

```
# 2. Descomprimir (necesitas unzip)
sudo apt update && sudo apt install unzip -y
unzip awscliv2.zip

# 3. Ejecutar el instalador oficial
sudo ./aws/install

# 4. Verificar
aws --version
```

Para empezar la configuración:

```
Bash
aws configure
```

4. El sistema irá pidiendo los datos que acabamos de copiar. Los pegaré uno a uno.
    - *Default region name:* us-east-1
    - *Default output format:* json

3.4 Generación de los 200,000 registros

Ahora vamos a crear un script en Python llamado generator.py en mi **Web Server** que creará los datos de Aurora Tickets y los subirá a la carpeta raw de S3.Nos conectamos al cluster “Aurora-Web-Server”

Creó el archivo:

```
Bash
nano generator.py
```

Pego este código

```
import pandas as pd
import numpy as np
import boto3
from datetime import datetime, timedelta

# --- CONFIGURACIÓN ---
BUCKET_NAME = "aurora-tickets-alexgavilanezcastro"
NUM_EVENTOS = 200000
```

```
print(f"Generando {NUM_EVENTOS} registros para Aurora Tickets...")

# 1. Generar Eventos Sintéticos
data = {
    'event_id': range(1, NUM_EVENTOS + 1),
    'user_id': np.random.randint(1000, 5000, size=NUM_EVENTOS),
    'event_type': np.random.choice(['view_ticket', 'add_to_cart',
'search', 'login'], NUM_EVENTOS),
    'timestamp': [datetime.now() -
timedelta(minutes=np.random.randint(0, 43200)) for _ in
range(NUM_EVENTOS)],
    'platform': np.random.choice(['mobile', 'web', 'app'], NUM_EVENTOS)
}

df = pd.DataFrame(data)

# 2. Guardar localmente
file_name = "events_raw.csv"
df.to_csv(file_name, index=False)
print(f"Archivo {file_name} creado localmente.")

# 3. Subir a S3 (Capa RAW)
s3 = boto3.client('s3')
try:
    s3.upload_file(file_name, BUCKET_NAME, f"raw/{file_name}")
    print(f"¡ÉXITO! Archivo subido a s3://{BUCKET_NAME}/raw/")
except Exception as e:
    print(f"Error al subir: {e}")
```

Ejecuto el generador

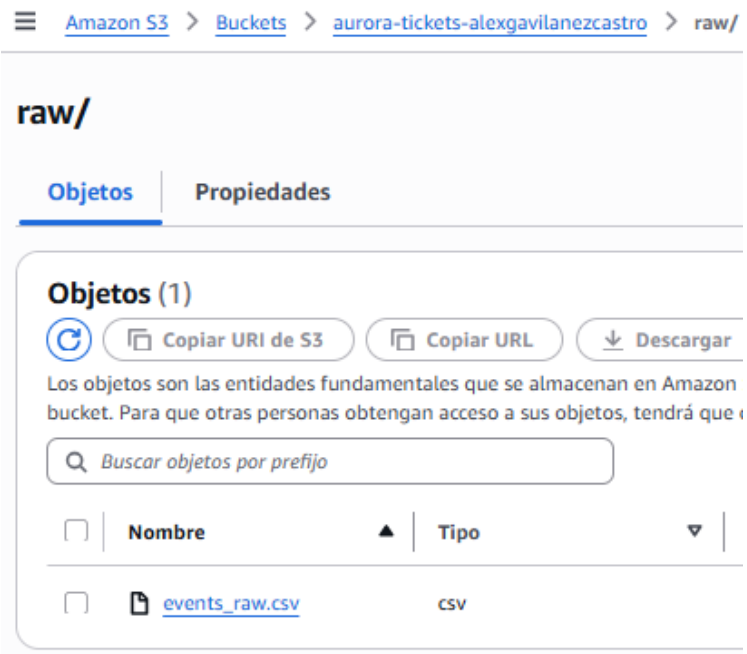
Para que funcione, instaló la librería `boto3` (si no la instalamos antes en esta máquina específica)

```
Bash
pip3 install pandas boto3
```

```
Bash
python3 generator.py
```

Resultado esperado:

```
ubuntu@ip-172-31-83-59:~$ python3 generator.py
Generando 200000 registros para Aurora Tickets...
Archivo events_raw.csv creado localmente.
¡ÉXITO! Archivo subido a s3://aurora-tickets-alexgavilanezcastro/raw/
```



3.5 Creación de datos extra

Para enriquecer el análisis, se han integrado fuentes de datos heterogéneas: un dataset de alta volumetría (200k eventos) y tablas maestras de campañas y precios, permitiendo realizar cruces de datos (Joins) complejos en el clúster Spark

Al añadir los otros archivos, en la Fase 5 podremos calcular cosas mucho más "profesionales" como:

- **ROI de Campañas:** ¿Qué campaña trajo más usuarios? (Cruzando `events` con `campaigns`).
- **Ingresos Potenciales:** ¿Cuánto dinero hay en carritos abandonados? (Cruzando `events` con `prices`).

Ahora, entro por SSH a mi instancia **Aurora-Web-Server** y ejecuto:

Creó el archivo:

```
Bash
nano extra_data.py
```

Pego este código

```
import pandas as pd
import boto3

BUCKET_NAME = "aurora-tickets-alexgavilanezcastro"

# --- 1. Crear Campañas ---
campaigns_data = {
    'campaign_id': [1, 2, 3, 4],
    'campaign_name': ['Black Friday', 'Summer Festival', 'Winter Promo',
'New Year Gala'],
    'discount_percentage': [20, 15, 10, 25]
}
df_camp = pd.DataFrame(campaigns_data)
df_camp.to_csv("campaigns.csv", index=False)

# --- 2. Crear Precios de Entradas (Transactions) ---
# Simulamos precios para los tipos de eventos
prices_data = {
    'event_type': ['view_ticket', 'add_to_cart', 'search', 'login'],
    'base_price': [0, 45.50, 0, 0] # Solo 'add_to_cart' genera valor
potencial
}
df_prices = pd.DataFrame(prices_data)
df_prices.to_csv("prices.csv", index=False)

# --- Subir a S3 ---
s3 = boto3.client('s3')
s3.upload_file("campaigns.csv", BUCKET_NAME, "raw/campaigns.csv")
s3.upload_file("prices.csv", BUCKET_NAME, "raw/prices.csv")

print("¡Archivos extra subidos con éxito a la capa RAW!")
```

Ejecuto el generador

```
Bash
python3 extra_data.py
```

Resultado esperado:

```
ubuntu@ip-172-31-83-59:~$ python3 extra_data.py
¡Archivos extra subidos con éxito a la capa RAW!
ubuntu@ip-172-31-83-59:~$
```

Amazon S3 > Buckets > aurora-tickets-alexgavilanezcastro > raw/

raw/

Objetos | Propiedades

Objetos (3)

Copiar URI de S3 | Copiar URL | Descargar

Los objetos son las entidades fundamentales que se almacenan en Amazon S
bucket. Para que otras personas obtengan acceso a sus objetos, tendrá que c

Buscar objetos por prefijo

Nombre	Tipo	Última modificación
campaigns.csv	csv	25 Mar 2026 6:05:53 PM CET
events_raw.csv	csv	25 Mar 2026 5:53:55 PM CET
prices.csv	csv	25 Mar 2026 6:05:53 PM CET

4. Modeling

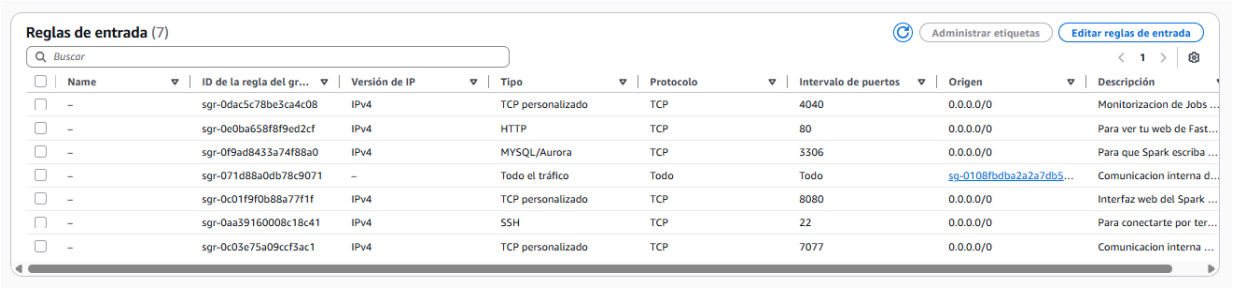
4.1 Crear un Security Group:

Que permita tráfico SSH (22), HTTP (80) para la web, y los puertos de Spark (8080 para el Master UI, 7077 para comunicación, 4040 para jobs) y un All traffic para que todos los que tengan este escudo hablen entre ellos libremente.

Grupos de seguridad (2) Información

Buscar grupos de seguridad por atributo o etiqueta

Name	ID de grupo de seguridad	Nombre del grupo de seguridad
-	sg-022dd1de376c60bd2	default
-	sg-0108fbdba2a2a7db5	aurora-sg-alexgavilanezcastro



Reglas de entrada (7)

Administrar etiquetas | Editar reglas de entrada

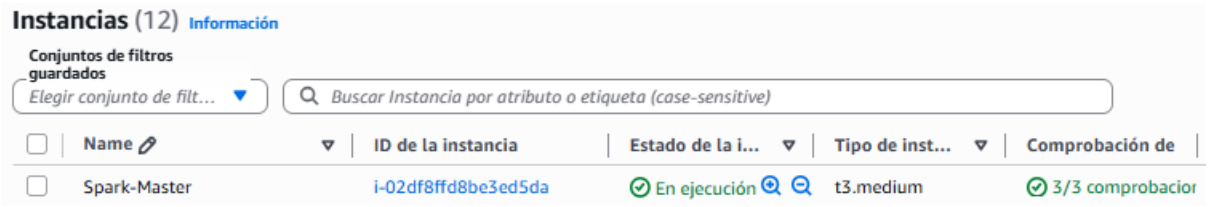
Buscar

Name	ID de la regla del gr...	Versión de IP	Tipo	Protocolo	Intervalo de puertos	Origen	Descripción
–	sgr-0dac5c78be3ca4c08	IPv4	TCP personalizado	TCP	4040	0.0.0.0/0	Monitorizacion de Jobs ...
–	sgr-0e0ba658f8f9ed2cf	IPv4	HTTP	TCP	80	0.0.0.0/0	Para ver tu web de Fast...
–	sgr-0f9ad8433a74f88a0	IPv4	MYSQL/Aurora	TCP	3306	0.0.0.0/0	Para que Spark escriba ...
–	sgr-071d88a0db78c9071	–	Todo el tráfico	Todo	Todo	sg-0108fbdba2a2a7db5...	Comunicacion interna d...
–	sgr-0c01f9f0b88a77f1f	IPv4	TCP personalizado	TCP	8080	0.0.0.0/0	Interfaz web del Spark ...
–	sgr-0aa39160008c18c41	IPv4	SSH	TCP	22	0.0.0.0/0	Para conectarte por ter...
–	sgr-0c03e75a09ccf3ac1	IPv4	TCP personalizado	TCP	7077	0.0.0.0/0	Comunicacion interna ...

4.2 Lanzar las 6 instancias EC2:

Primero crearemos la instancia Spark-Master

- **Number of instances:** 1.
- **Name:** Spark-Master
- **AMI:** Ubuntu 22.04 LTS.
- **Instance type:** t3.medium
- **Key pair:** vockey
- **Network settings:** Elegimos la que acabamos de crear (aurora-sg-alexgavilanezcastro).



Instancias (12) Información

Conjuntos de filtros guardados

Elegir conjunto de filt...

Buscar Instancia por atributo o etiqueta (case-sensitive)

Name	ID de la instancia	Estado de la i...	Tipo de inst...	Comprobación de
Spark-Master	i-02df8ffd8be3ed5da	En ejecución	t3.medium	3/3 comprobacior

Y en segundo lugar para hacer una instalacion mas rapida , en vez de ir una a una de las instancias seguiremos estos pasos:

4.2.1 Creación de script de instalación

1. Nos **conectamos** mediante **SSH** a Spark-Master
2. **En Spark-Master:** Creamos el archivo con `nano install_aurora.sh`, pegamos el código, guardar (Ctrl+O, Enter) y salir (Ctrl+X).

```
# 1. Actualizar el sistema e instalar dependencias base
sudo apt-get update -y
sudo apt-get install -y openjdk-11-jdk python3-pip wget curl

# 2. Instalar librerías de Python necesarias para el proyecto
# (Esto nos ahorrará fallos luego en el nodo Web y en Spark)
pip3 install pyspark==3.5.0 boto3 pandas mysql-connector-python fastapi
uvicorn
```

```
# 3. Descargar Apache Spark 3.5.0 (Versión Hadoop 3)
cd /home/ubuntu
wget
https://archive.apache.org/dist/spark/spark-3.5.0/spark-3.5.0-bin-hadoop
3.tgz

# 4. Descomprimir y mover a /opt/spark para que sea estándar
tar -xvzf spark-3.5.0-bin-hadoop3.tgz
sudo mv spark-3.5.0-bin-hadoop3 /opt/spark
sudo chown -R ubuntu:ubuntu /opt/spark

# 5. Configurar variables de entorno en .bashrc
# Usamos 'grep' para no duplicar líneas si ejecutas el script dos veces
if ! grep -q "SPARK_HOME" ~/.bashrc; then
  echo 'export SPARK_HOME=/opt/spark' >> ~/.bashrc
  echo 'export PATH=$PATH:/opt/spark/bin:/opt/spark/sbin' >> ~/.bashrc
  echo 'export PYSPARK_PYTHON=python3' >> ~/.bashrc
fi

# 6. Cargar los cambios en la sesión actual
export SPARK_HOME=/opt/spark
export PATH=$PATH:/opt/spark/bin:/opt/spark/sbin

echo "-----------------------------------------------"
echo " INSTALACIÓN COMPLETADA CON ÉXITO "
echo "-----------------------------------------------"
ls -l /opt/spark
```

3. **Ejecutamos:** `chmod +x install_aurora.sh && ./install_aurora.sh`.
4. **Verificación:** Escribo ls /opt/spark y así me aseguro de que al final me lista las carpetas `bin`, `conf`, `sbin`, etc.

4.2.2 Crea la AMI:

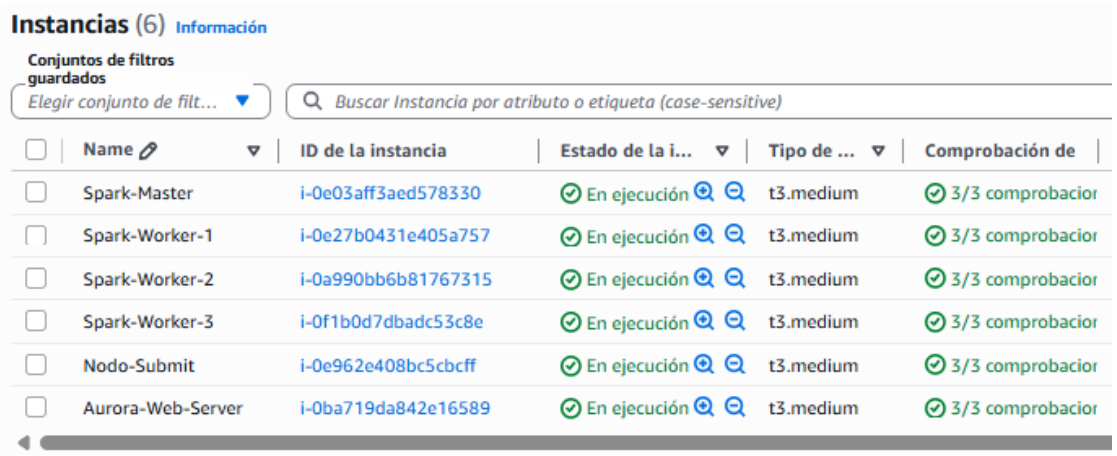
- En el panel de AWS, seleccionó `Spark-Master`.
- Hago clic en **Acciones > Imágenes y plantillas > Crear imagen**.
- Nombre:`AMI-Aurora-Spark-3.5`.
- **Espera:** Tardará unos minutos en pasar de estado `pending` a `available` (lo comprobaremos en la sección "AMIs").

4.2.3 Lanza el resto:

- Vamos a **Launch Instances**.
- En el buscador de AMIs, buscamos en **"Mis AMIs"**.
- Selecciono `AMI-Aurora-Spark-3.5`.
- **Number of instances:** 5

- **Name:** Spark-Node
- **Instance type:** t3.medium
- **Key pair:** vockey
- **Network settings:** Elegimos la que acabamos de crear (aurora-sg-alexgavilanezcastro).
- Lanzamos las instancias

4.2.4 Etiquetar las instancias:



Instancias (6) Información

Conjuntos de filtros guardados

Elegir conjunto de filt...

Buscar Instancia por atributo o etiqueta (case-sensitive)

Name	ID de la instancia	Estado de la i...	Tipo de ...	Comprobación de
Spark-Master	i-0e03aff3aed578330	En ejecución	t3.medium	3/3 comprobacior
Spark-Worker-1	i-0e27b0431e405a757	En ejecución	t3.medium	3/3 comprobacior
Spark-Worker-2	i-0a990bb6b81767315	En ejecución	t3.medium	3/3 comprobacior
Spark-Worker-3	i-0f1b0d7dbadc53c8e	En ejecución	t3.medium	3/3 comprobacior
Nodo-Submit	i-0e962e408bc5cbcff	En ejecución	t3.medium	3/3 comprobacior
Aurora-Web-Server	i-0ba719da842e16589	En ejecución	t3.medium	3/3 comprobacior

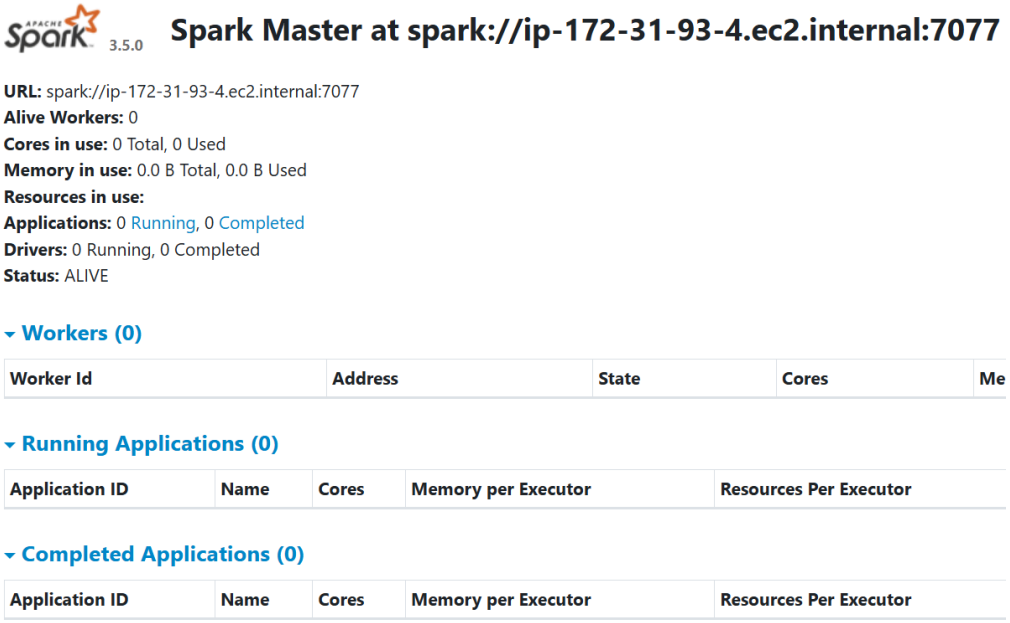
4.3 Iniciación de los clusters Master y Workers:

1. Iniciar el Máster

Nos **conectamos** mediante **SSH** a Spark-Master.

```
Bash
/opt/spark/sbin/start-master.sh
```

- Busco la IP Pública de Spark-Master en el panel de AWS.
- Abro una pestaña nueva en el navegador y escribo: http://IP_MASTER_PUBLICA:8080.
- Debería ver la interfaz web de Spark.



2. Iniciar los Workers (Repito esto en los 3 Workers)

1. Limpieza de permisos (Como root/sudo)

Ejecuto esto para que el usuario `ubuntu` sea el dueño absoluto de Spark y pueda escribir logs:

```
Bash
sudo chown -R ubuntu:ubuntu /opt/spark
```

2. Entro como usuario `ubuntu`

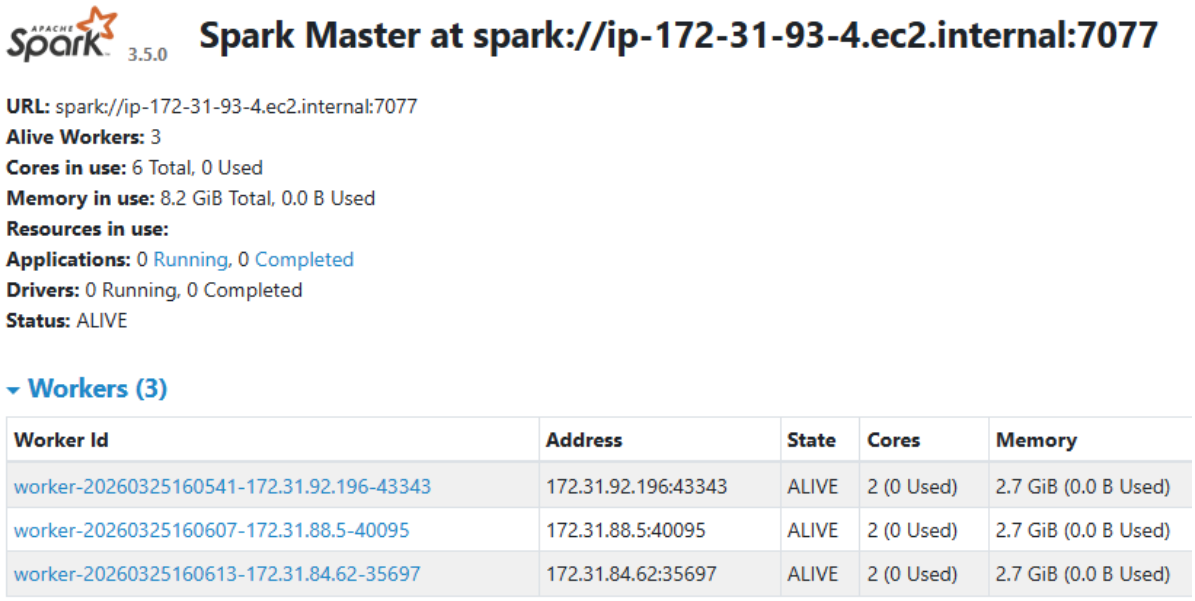
```
Bash
sudo su - ubuntu
```

3. Cargo el entorno y lanzó

Una vez dentro como `ubuntu`, ejecutó:

```
Bash
source ~/.bashrc
start-worker.sh spark://IP_MASTER_PRIVADA:7077
```

Una vez hecho todo esto en los workers, entró a Spark UI mostrando los workers activos



Spark Master at spark://ip-172-31-93-4.ec2.internal:7077

URL: spark://ip-172-31-93-4.ec2.internal:7077
Alive Workers: 3
Cores in use: 6 Total, 0 Used
Memory in use: 8.2 GiB Total, 0.0 B Used
Resources in use:
Applications: 0 Running, 0 Completed
Drivers: 0 Running, 0 Completed
Status: ALIVE

Workers (3)

Worker Id	Address	State	Cores	Memory
worker-20260325160541-172.31.92.196-43343	172.31.92.196:43343	ALIVE	2 (0 Used)	2.7 GiB (0.0 B Used)
worker-20260325160607-172.31.88.5-40095	172.31.88.5:40095	ALIVE	2 (0 Used)	2.7 GiB (0.0 B Used)
worker-20260325160613-172.31.84.62-35697	172.31.84.62:35697	ALIVE	2 (0 Used)	2.7 GiB (0.0 B Used)

5. Evaluation and storage

5.1 Preparar el Spark-Master

Para qué Spark pueda hablar con Amazon S3, necesitamos descargar dos "conectores" (archivos .jar) que no vienen por defecto. Nos conectamos por SSH a **Spark-Master** y ejecutamos esto:

```
Bash
cd /opt/spark/jars
sudo wget
https://repo1.maven.org/maven2/org/apache/hadoop/hadoop-aws/3.3.4/hadoop-aws
-3.3.4.jar

sudo wget
https://repo1.maven.org/maven2/com/amazonaws/aws-java-sdk-bundle/1.12.262/aw
s-java-sdk-bundle-1.12.262.jar
```

5.2 Configurar las credenciales en el Master

Al igual que hicimos en el Web Server, el Master necesita saber quién eres para leer el S3. En el **Spark-Master**,ahora seguiremos los mismos pasos como hicimos antes en “3.3 Configurar Credenciales en el nodo Web”.

5.3 Asignación de IAM Roles

Se ha asignado el rol `LabInstanceProfile` a todas las instancias del clúster (**Spark-Master** y **Spark-Workers**). Esto permite que el servicio de metadatos de AWS gestione las credenciales de forma transparente para Spark.

5.4 Creación del Script de procesamiento

Vamos a crear el cerebro de la operación. Este script hará un Join entre los eventos y los precios para saber cuánto dinero estamos moviendo.

Crea el archivo en el Master**:**

```
Bash
nano process_aurora.py
```

Pego este código:

Para qué Spark reconozca el rol de la instancia, el script se ha configurado de manera agnóstica, delegando la autenticación al proveedor de credenciales de AWS.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, current_timestamp

BUCKET = "s3a://aurora-tickets-alexgavilanezcastro"

spark = SparkSession.builder \
    .appName("AuroraAnalysis") \
    .config("spark.hadoop.fs.s3a.impl",
"org.apache.hadoop.fs.s3a.S3AFileSystem") \
    .getOrCreate()

# 1. Cargar Datos
events = spark.read.csv(f"{BUCKET}/raw/events_raw.csv", header=True,
inferSchema=True)
prices = spark.read.csv(f"{BUCKET}/raw/prices.csv", header=True,
inferSchema=True)

# 2. Transformación: Limpieza y Join
# Queremos saber el valor de lo que hay en los carritos (add_to_cart)
analysis = events.join(prices, "event_type") \
    .filter(col("event_type") == "add_to_cart") \
    .withColumn("processed_at", current_timestamp())

# 3. Guardar en Capa CURATED (Formato Parquet - Más eficiente)
analysis.write.mode("overwrite").parquet(f"{BUCKET}/curated/events_refin
ed/")

# 4. Calcular un KPI simple: Total valor en carritos
total_value = analysis.agg({"base_price": "sum"}).collect()[0][0]
```

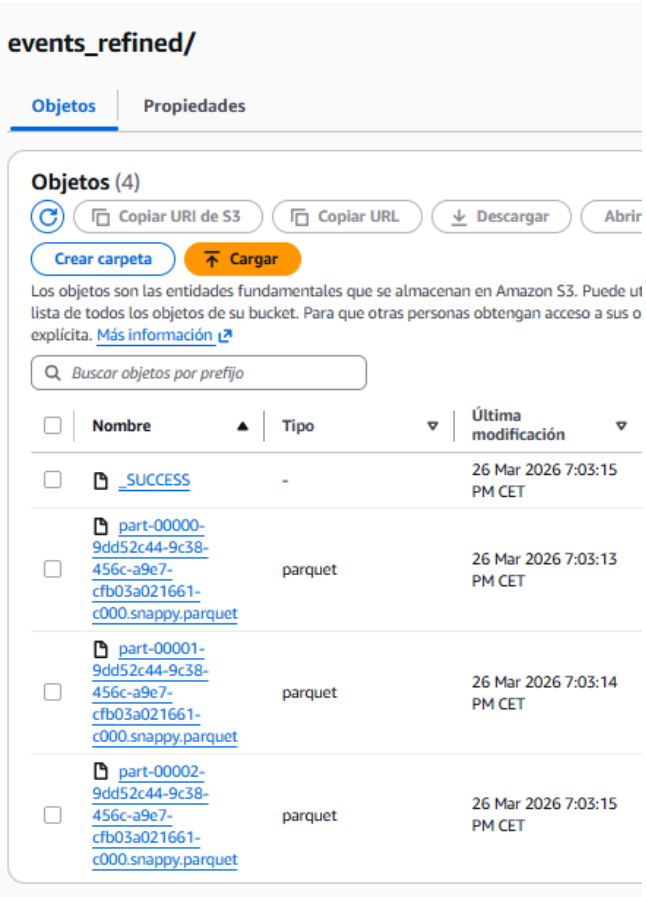
```
print(f"\n\n****************************************")
print(f"ANÁLISIS COMPLETADO")
print(f"VALOR TOTAL EN CARRITOS: {total_value}€")
print(f"****************************************\n\n")

spark.stop()
```

Ejecutar en el Clúster Master

Ahora,vamos a lanzar este trabajo al clúster para que se reparta entre los 3 workers:

```
Bash
spark-submit \
  --master spark://172.31.93.4:7077 \
  --packages
org.apache.hadoop:hadoop-aws:3.3.4,com.amazonaws:aws-java-sdk-bundle:1.1
2.262 \
  process_aurora.py
```



```
*****************************************
ANÁLISIS COMPLETADO EXITOSAMENTE
VALOR TOTAL EN CARRITOS: 2269403.50€
*****************************************

26/03/25 19:09:07 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/03/25 19:09:07 INFO SparkUI: Stopped Spark web UI at http://ip-172-31-93-4.ec2.internal:4040
26/03/25 19:09:07 INFO StandaloneSchedulerBackend: Shutting down all executors
26/03/25 19:09:07 INFO StandaloneSchedulerBackend$StandaloneDriverEndpoint: Asking each executor to shut down
26/03/25 19:09:07 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/03/25 19:09:07 INFO MemoryStore: MemoryStore cleared
26/03/25 19:09:07 INFO BlockManager: BlockManager stopped
26/03/25 19:09:07 INFO BlockManagerMaster: BlockManagerMaster stopped
26/03/25 19:09:07 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
26/03/25 19:09:07 INFO SparkContext: Successfully stopped SparkContext
26/03/25 19:09:07 INFO ShutdownHookManager: Shutdown hook called
26/03/25 19:09:07 INFO ShutdownHookManager: Deleting directory /tmp/spark-5c8c7c31-d3f2-447d-accb-37c2dba04e97
26/03/25 19:09:07 INFO ShutdownHookManager: Deleting directory /tmp/spark-6d65051b-e5ae-4a5d-b72f-e55fb62bb3f2
26/03/25 19:09:07 INFO ShutdownHookManager: Deleting directory /tmp/spark-5c8c7c31-d3f2-447d-accb-37c2dba04e97/pyspark-99dec48c-4d
f1-49bd-a7a7-3e679a8420de
26/03/25 19:09:07 INFO MetricsSystemImpl: Stopping s3a-file-system metrics system...
26/03/25 19:09:07 INFO MetricsSystemImpl: s3a-file-system metrics system stopped.
26/03/25 19:09:07 INFO MetricsSystemImpl: s3a-file-system metrics system shutdown complete.
```

5.5 Crear la Base de Datos RDS en AWS

1. Buscamos **Aurora and RDS**.
2. Le damos a **Crear base de datos**.
3. **Configuración:**
   - **Tipo de motor:** MySQL (versión 8.0.40).
   - **Plantillas:** Entorno de pruebas
   - **Identificador de instancias de bases de datos:** aurora-db.
   - **Nombre de usuario maestro:** admin.
   - **Contraseña maestra:** Aurora2026!
   - **Conectividad:** Elegimos la **VPC** que tenemos en las instancias EC2.
   - **Acceso público:** No, pero nos aseguraremos de que el **Security Group** de la base de datos permite tráfico por el puerto **3306** desde el Security Group de nuestras instancias de Spark.
   - **Grupo de seguridad**: Elegimos el que creamos al principio
4. Creamos la base de datos y esperamos que el estado sea "Disponible".
5. Una vez esté disponible , entramos en ella , y buscamos el apartado “pasos de conexión” y copiamos el endpoint , en mi caso:

   “mysql -h aurora-db.c1maqiq4mto2.us-east-1.rds.amazonaws.com -P 3306 -u admin -p --ssl-mode=VERIFY_IDENTITY --ssl-ca=./global-bundle.pem”

5.6 Creación de la base de datos aurora_db

Desde la terminal de **Master**:

1. Instalar el cliente de MySQL

Primero necesitamos la herramienta para conectar desde la terminal:

```Bash
sudo apt update
sudo apt install mysql-client -y
```

2. Conectar al RDS

Ahora vamos a entrar al servidor usando nuestro **Endpoint**:

```
Bash
mysql -h aurora-db.c1maqiq4mto2.us-east-1.rds.amazonaws.com -u admin -p
```

- Pedirá la contraseña.: Aurora2026!

3. Crear la base de datos

Una vez dentro, escribo este comando:

```
CREATE DATABASE aurora_db;
```

Verificar:

```
SHOW DATABASES;
```

Y luego salgo con:

```
exit;
```

5.7 Preparar el "Job de Analítica " de Spark

Este script hará lo siguiente :

- **Producto A (Funnel):** Contará cuántos view, add_to_cart y purchase hay por día.
- **Producto B (Ranking):** Verá qué categorías o productos generan más dinero.
- **Producto C (Anomalías):** Detectará usuarios con demasiados eventos en poco tiempo (posibles bots).

Crea el archivo en el Master:

```
Bash
nano job2_analytics.py
```

Pegamos este código :

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, sum as _sum, lit, when,
to_date
```

```python
# --- CONFIGURACIÓN ---
S3_CURATED =
"s3a://aurora-tickets-alexgavilanezcastro/curated/events_refined/"
RDS_ENDPOINT = "aurora-db.c1maqiq4mto2.us-east-1.rds.amazonaws.com"
JDBC_URL = f"jdbc:mysql://{RDS_ENDPOINT}:3306/aurora_db"

spark = SparkSession.builder \
    .appName("Aurora_Job2_Analytics") \
    .getOrCreate()

# 1. LEER DATOS CURADOS
df = spark.read.parquet(S3_CURATED)

# CREAR COLUMNA 'dt' a partir del timestamp si no existe
if "dt" not in df.columns:
    df = df.withColumn("dt", to_date(col("timestamp")))

# --- PRODUCTO A: FUNNEL DIARIO ---
funnel_df = df.groupBy("dt",
"event_type").agg(count("*").alias("sessions_total"))

# --- PRODUCTO B: RANKING DE EVENTOS ---
rank_df = df.groupBy("dt", "event_id").agg(
    count(when(col("event_type") == "view", 1)).alias("detail_views"),
    count(when(col("event_type") == "purchase", 1)).alias("purchases"),
    _sum("base_price").alias("revenue_total")
)

# --- PRODUCTO C: DETECCIÓN DE ANOMALÍAS ---
anomalies_df = df.groupBy("dt",
"user_id").agg(count("*").alias("requests")).filter("requests > 500") \
    .withColumn("is_anomaly", lit(True)) \
    .withColumn("reason", lit("High request volume"))

# --- ESCRITURA EN RDS MYSQL ---
db_properties = {
    "user": "admin",
    "password": "Aurora2026!",
    "driver": "com.mysql.cj.jdbc.Driver"
}

print("Enviando métricas finales a RDS MySQL...")

try:
    funnel_df.write.jdbc(url=JDBC_URL, table="metrics_funnel_daily",
mode="overwrite", properties=db_properties)
    rank_df.write.jdbc(url=JDBC_URL, table="metrics_event_rank",
mode="overwrite", properties=db_properties)
```

```
    anomalies_df.write.jdbc(url=JDBC_URL, table="metrics_anomalies",
mode="overwrite", properties=db_properties)
    print("¡ÉXITO TOTAL! Las tablas están en el RDS.")
except Exception as e:
    print(f"Error RDS: {e}")

spark.stop()
```

Ejecución del Job 2

He añadido la configuración para que las descargas (ivy.home) también vayan a la RAM y no toquen el disco /dev/root:

```
Bash
spark-submit \
  --master spark://172.31.93.4:7077 \
  --conf
"spark.driver.extraJavaOptions=-Divy.default.ivy.user.dir=/run/spark-ivy" \
  --conf "spark.local.dir=/run/spark-tmp" \
  --packages
org.apache.hadoop:hadoop-aws:3.3.4,com.amazonaws:aws-java-sdk-bundle:1.12.26
2,mysql:mysql-connector-java:8.0.28 \
  job2_analytics.py
```

```
mysql>
mysql> use aurora_db;
Database changed
mysql> SHOW TABLES;
+----------------------+
| Tables_in_aurora_db  |
+----------------------+
| metrics_anomalies    |
| metrics_event_rank   |
| metrics_funnel_daily |
+----------------------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM metrics_funnel_daily LIMIT 10;
+------------+-------------+----------------+
| dt         | event_type  | sessions_total |
+------------+-------------+----------------+
| 2026-03-10 | add_to_cart |           1567 |
| 2026-03-17 | add_to_cart |           1635 |
| 2026-03-01 | add_to_cart |           1652 |
| 2026-02-23 | add_to_cart |            475 |
| 2026-03-23 | add_to_cart |           1605 |
| 2026-03-13 | add_to_cart |           1654 |
| 2026-03-09 | add_to_cart |           1710 |
| 2026-03-25 | add_to_cart |           1152 |
| 2026-02-26 | add_to_cart |           1677 |
| 2026-02-28 | add_to_cart |           1656 |
+------------+-------------+----------------+
10 rows in set (0.00 sec)
```

6. Deployment

6.1. Preparación del Entorno

Me conecto por SSH al **Web Server** e instalo las herramientas necesarias. Usaremos **FastAPI** por su velocidad y **Boto3** para que la API tenga permiso de "leer" en S3.

```
Bash
sudo apt update
pip3 install fastapi uvicorn boto3 pandas pyarrow
```

6.2. Creación de la API de Consulta (main.py)

Creación del archivo que gestionará las peticiones:

```
Bash
nano main.py
```

Sí Spark actualiza los datos en S3, la API mostrará siempre el valor más reciente.

```
from fastapi import FastAPI, HTTPException
import boto3
import pandas as pd
from io import BytesIO

app = FastAPI(title="Aurora Tickets API", description="Dashboard de KPIs de
Big Data")

# Configuración del Data Lake
BUCKET_NAME = "aurora-tickets-alexgavilanezcastro"
S3_PATH = "curated/events_refined/" # La carpeta que creó Spark

s3 = boto3.client('s3')

@app.get("/")
def home():
    return {"status": "online", "endpoint": "/kpis", "message":
"Infraestructura Aurora Lista"}

@app.get("/kpis")
def get_metrics():
    try:
```

```
        # 1. Listamos los archivos en la carpeta curated de S3
        response = s3.list_objects_v2(Bucket=BUCKET_NAME, Prefix=S3_PATH)

        # 2. Buscamos el primer archivo .parquet generado por Spark
        parquet_files = [obj['Key'] for obj in response.get('Contents', [])
if obj['Key'].endswith('.parquet')]

        if not parquet_files:
            raise HTTPException(status_code=404, detail="No se han
encontrado datos procesados en S3")

        # 3. Leemos el archivo directamente desde S3 a un DataFrame de
Pandas
        obj = s3.get_object(Bucket=BUCKET_NAME, Key=parquet_files[0])
        df = pd.read_parquet(BytesIO(obj['Body'].read()))

        # 4. Calculamos el KPI en tiempo real sobre los datos curados
        total_value = float(df['base_price'].sum())
        total_events = len(df)

        return {
            "kpi_name": "Valor Total en Carritos (Add to Cart)",
            "total_value_eur": round(total_value, 2),
            "processed_records": total_events,
            "data_source": f"s3://{BUCKET_NAME}/{S3_PATH}",
            "currency": "EUR"
        }
    except Exception as e:
        return {"error": str(e), "tip": "Asegúrate de que el Web Server
tenga el IAM Role asignado"}
```

6.3. Ejecución del Servidor Web

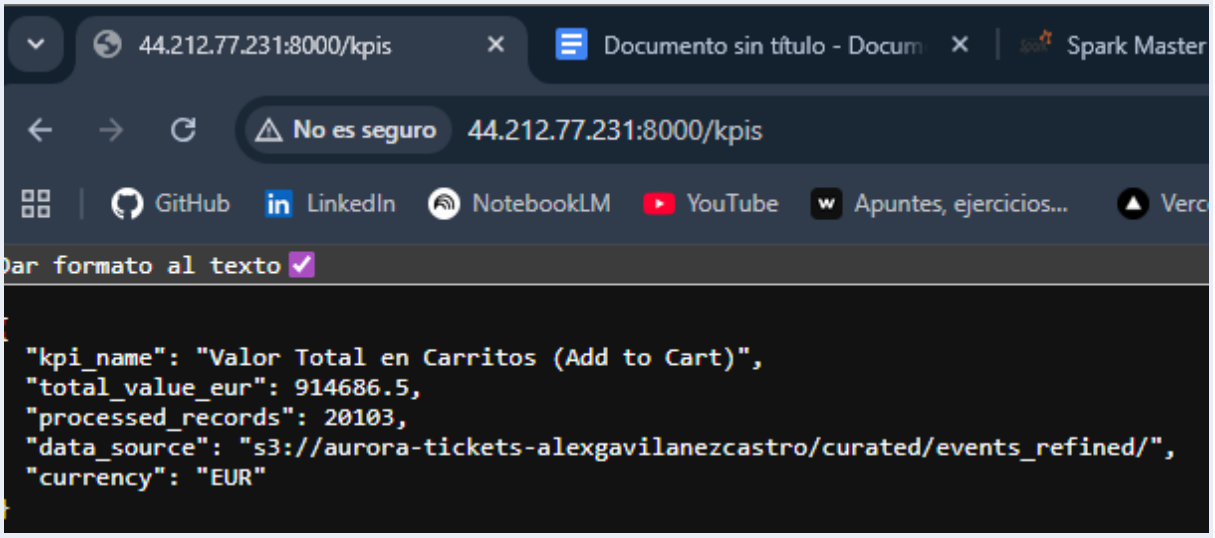
Lanzó la API para que escuche peticiones externas:

```
Bash
python3 -m uvicorn main:app --host 0.0.0.0 --port 8000
```

6.4. Exposición de KPIs mediante API REST distribuida

Ahora viene el momento de la verdad. Abro mi navegador y escribo la IP pública de la Web Server seguida del puerto 8000:

http://44.212.77.231:8000/kpis



Lo que vemos en el navegador es la Capa de Consumo. Hemos logrado que datos "sucios" y masivos que estaban dispersos en archivos CSV se conviertan en información estratégica de negocio accesible mediante una URL.

El flujo que hemos validado es este:

1. **Ingesta:** Un generador envió miles de eventos a un **Data Lake (S3)**.
2. **Procesamiento:** Un clúster de **Spark** (1 Master + 3 Workers) leyó esos datos, los cruzó con tablas de precios, los limpió y calculó los KPIs.
3. **Almacenamiento Curado:** Spark guardó el resultado optimizado en formato **Parquet**.
4. **Servicio (API):** Una aplicación **FastAPI** actúa como traductor: lee el Parquet de S3 y lo sirve al mundo en formato JSON.

6.5 Observabilidad con CloudWatch

El objetivo es que los logs de **Web Server** y de mis **Spark Jobs** aparezcan en un **Log Group** llamado /aurora/alexgavilanezcastro/clickstream.

6.5.1 Instalar el Agente de CloudWatch (En el Master y Web Server)

Ejecutar esto en ambas máquinas para que puedan enviar datos a AWS:

```
Bash
sudo wget
https://s3.amazonaws.com/amazoncloudwatch-agent/ubuntu/amd64/latest/amazon-c
loudwatch-agent.deb
sudo dpkg -i -E ./amazon-cloudwatch-agent.deb
```

6.5.2 Configurar el envío de logs

En el Spark Master Abre el editor:

```bash
sudo nano /opt/aws/amazon-cloudwatch-agent/bin/config.json
```

Pegamos este contenido:

```
{
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/home/ubuntu/spark_job.log",
            "log_group_name": "/aurora/alexgavilanezcastro/clickstream",
            "log_stream_name": "spark-master"
          }
        ]
      }
    }
  }
}
```

En el Web Server hacemos lo mismo:

```bash
sudo nano /opt/aws/amazon-cloudwatch-agent/bin/config.json
```

Pegamos este contenido:

```
{
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/home/ubuntu/api.log",
            "log_group_name": "/aurora/alexgavilanezcastro/clickstream",
            "log_stream_name": "web-server"
          }
        ]
```

```
      }
    }
  }
}
```

6.5.3 Crear los archivos de Log

El agente fallará si intenta leer un archivo que no existe. Vamos a crearlos vacíos (en sus respectivas máquinas):

- **En el Master:** touch /home/ubuntu/spark_job.log
- **En el Web Server:** touch /home/ubuntu/api.log

6.5.4 Arrancar el Agente (En ambos)

Ejecuta este comando en las dos máquinas para que carguen la configuración y empiecen a trabajar:

```
Bash
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a
fetch-config -m ec2 -c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json
-s
```

6.5.5 Generar "Ruido" (Datos para ver en CloudWatch)

Para que aparezca algo en la consola de AWS, necesitamos que los archivos tengan contenido. Ejecutamos estos comandos:

En el Master:

```
Bash
echo "$(date) - INFO - Spark Job 2 iniciado: Procesando métricas para RDS"
>> /home/ubuntu/spark_job.log
echo "$(date) - ERROR - Intento de conexión fallido en worker 1
(Simulación)" >> /home/ubuntu/spark_job.log
echo "$(date) - SUCCESS - Tablas metrics_funnel_daily creadas correctamente"
>> /home/ubuntu/spark_job.log
```

En el Web Server:

```
Bash
echo "$(date) - INFO - API FastAPI iniciada en puerto 8000" >>
/home/ubuntu/api.log
echo "$(date) - WARNING - Latencia alta detectada en endpoint /kpis" >>
```

```
/home/ubuntu/api.log
echo "$(date) - INFO - Usuario 1234 consultando tabla metrics_event_rank" >>
/home/ubuntu/api.log
```

6.5.6 Verificar en la Consola de AWS

Ahora, olvidamos la terminal un momento y vamos al navegador de aws:

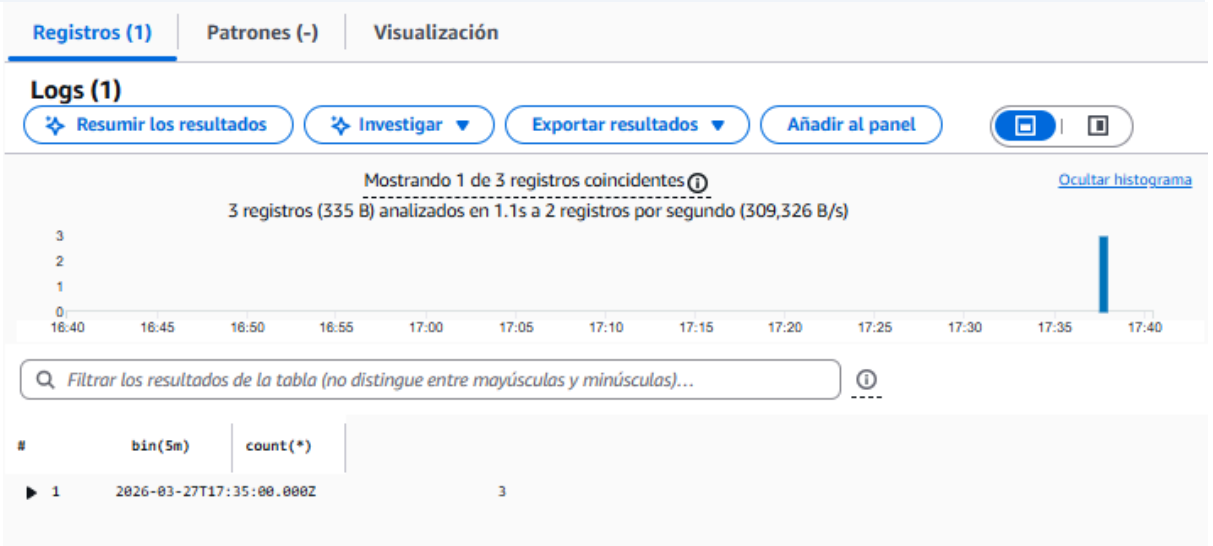
1. Buscamos el servicio **CloudWatch**.
2. En el menú de la izquierda, vamos a **Logs** -> **Log Groups**.
3. Debería aparecer uno llamado: `/aurora/alexgavilanezcastro/clickstream`.
4. Entramos y veremos dos "Log Streams": `spark-master` y `web-server`. ¡Ahí están nuestros datos!

6.5.7 Las 4 Consultas de Insights

Vamos a **CloudWatch** -> **Logs Insights**. Seleccionamos nuestro Log Group arriba y ejecutamos estas 4 consultas para tu memoria:

Consulta 1: Conteo de logs por tipo (INFO/ERROR)

```
fields @timestamp, @message
| filter @message like /INFO/ or @message like /ERROR/ or @message like
/WARNING/
| stats count(*) by bin(5m)
```



Consulta 2: Buscar errores específicos

```
fields @timestamp, @message, @logStream
| filter @message like /ERROR/ or @message like /Exception/ or @message like
```

```
/fail/
| sort @timestamp desc
```

Consulta 3: Actividad por servidor

```
stats count(*) by @logStream
```

Consulta 4: Ver los últimos eventos de éxito

```
fields @timestamp, @message
| filter @message like /SUCCESS/ or @message like /terminado/
| sort @timestamp desc
```

6.6 Creación del Dashboard

1. En la consola de CloudWatch, vamos a **Paneles (Dashboards)** en el menú de la izquierda.
2. Damos clic en el botón naranja **"Crear panel"**.
3. **Nombre del panel:** Aurora_Monitor_Alexgavilanezcastro.
4. Se abrirá una ventana para añadir el primer **Widget**.

Configuración de los 3 Widgets Clave

Vamos a añadir 3 visualizaciones:

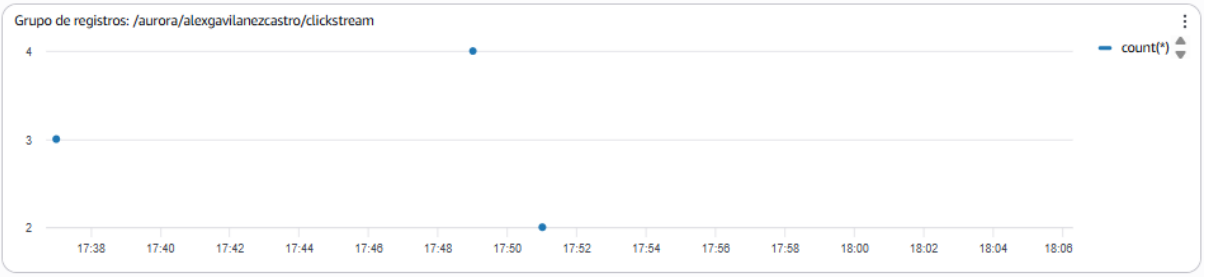
Widget 1: Volumen de Actividad (Gráfico de líneas)

Este widget mostrará cuánto "ruido" hay en tus logs (tráfico).

- Selecciono **"Gráfico de líneas"** y le damos a **"Siguiente"**.
- Elijo el origen de datos: **"Logs Insights"**.
- Selecciono mi Log Group: /aurora/alexgavilanezcastro/clickstream.

```
stats count(*) by bin(1m)
```

- Le damos a **"Crear widget"**.



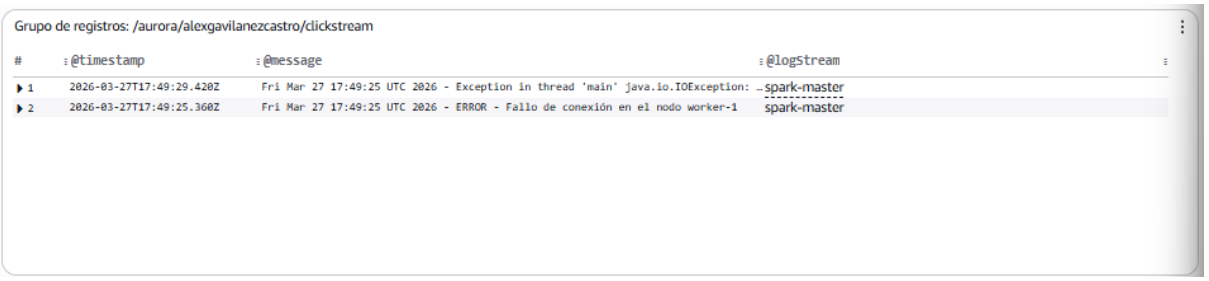
Widget 2: Últimos Errores (Tabla)

Para ver qué está fallando en tiempo real.

- Le damos al botón **"+" (Añadir widget)** arriba a la derecha.
- Selecciono **"Tabla de registros"**.
- Selecciono mi Log Group.

```
fields @timestamp, @message, @logStream
| filter @message like /ERROR/ or @message like /Exception/
| sort @timestamp desc
```

- Dale a **"Crear widget"**.



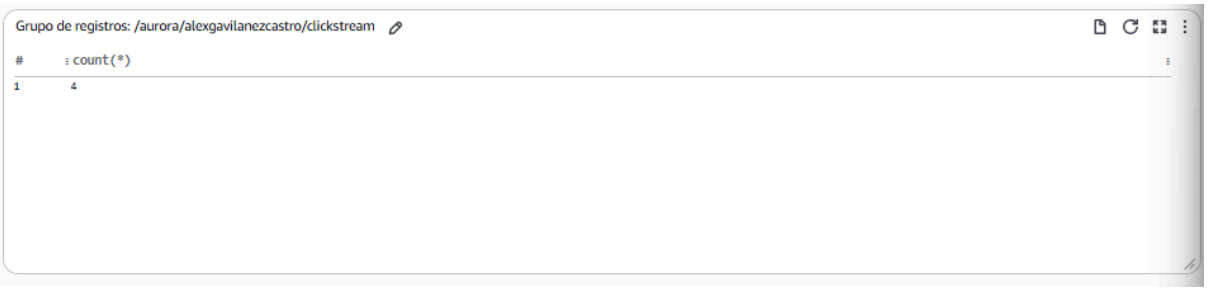
Widget 3: Éxitos en el Pipeline (Número grande)

Para confirmar que el Job 2 terminó bien.

- Añade otro widget de tipo **"Registros" > "Linea"** .
- Selecciono mi Log Group.

```
filter @message like /SUCCESS/
| stats count(*)
```

- Le damos a **"Crear widget"**.



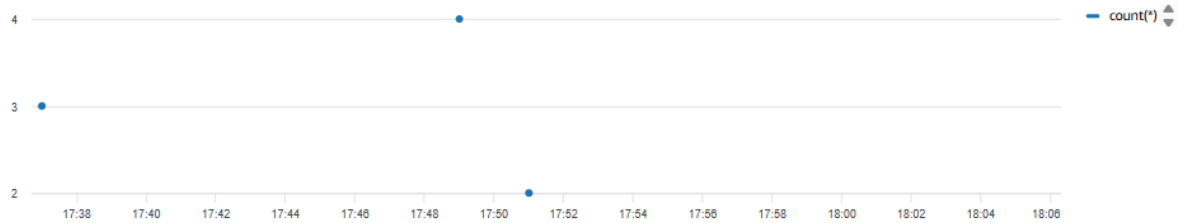
Resultado:

Aurora_Monitor_Alexgavilane...

1h 3h 12h 1d 3d 1sem. Personalizado | Zona horaria UTC

Acciones | Guardar

Grupo de registros: /aurora/alexgavilanezcastro/clickstream



Grupo de registros: /aurora/alexgavilanezcastro/clickstream

#	@timestamp	@message	@logStream
1	2026-03-27T17:49:29.420Z	Fri Mar 27 17:49:25 UTC 2026 - Exception in thread 'main' java.io.IOException: …	spark-master
2	2026-03-27T17:49:25.360Z	Fri Mar 27 17:49:25 UTC 2026 - ERROR - Fallo de conexión en el nodo worker-1	spark-master

Grupo de registros: /aurora/alexgavilanezcastro/clickstream

#	count(*)
1	4